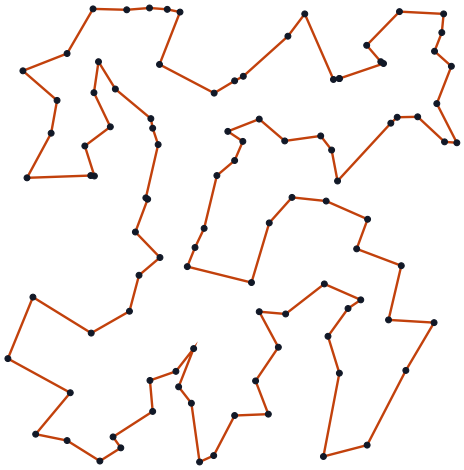
Project report

Solving the Travelling Salesman Problem with Ant Colony Optimisation

Design, implementation and evaluation of `aco-tsp`



Dmytro Palahin

Version 1.0 · May 2023
Version 2.0 · November 2026

Revised and extended edition of a report written in February 2023, with a classmate, for the course *Projet Informatique* (CP2I2, Sup Galilée, Université Sorbonne Paris Nord), taught by Frédéric Roupin.

Abstract

This report describes `aco-tsp`, a C11 library and command-line tool that searches for short tours of the symmetric travelling salesman problem with ant colony optimisation. It states the problem and the representation of its solutions, derives the Ant System algorithm and three classic refinements, explains how they are implemented and verified, and evaluates them experimentally. On 500 random instances small enough for the optimum to be proven, Ant System finds it in 89 % to 100 % of the runs, and in every run once it is combined with a 2-opt local search. On the instances of up to 100 cities supplied with the original assignment, that hybrid produces tours 2.9 % to 11.9 % shorter than a strong deterministic baseline, in about half a second. The project started as second-year coursework; the report also documents what was wrong with that first version and how the rewrite addresses it.

Contents

1 Introduction **2**

2 The problem **2**
2.1 Statement . . . 2
2.2 Objective function . . . 3
2.3 Representation of a solution . . . 3
2.4 Input format . . . 3

3 Ant colony optimisation **3**
3.1 Principle . . . 3
3.2 Ant System . . . 4
3.3 Three refinements . . . 5
3.4 Parameters . . . 6

4 Implementation **6**
4.1 Architecture . . . 6
4.2 The inner loop . . . 7
4.3 Numerical robustness . . . 7
4.4 Randomness and reproducibility . . . 8
4.5 Reference solvers . . . 8
4.6 From the 2023 prototype to version 2 . . . 8
4.7 Verification . . . 9

5 Experiments **10**
5.1 Protocol . . . 10
5.2 Validation against proven optima . . . 10
5.3 Tuning the parameters . . . 10
5.4 Results on the instances of the assignment . . . 11
5.5 Convergence . . . 12
5.6 Running time . . . 14

6 Conclusion **14**

References **14**

A Using the program **16**

1 Introduction

A delivery driver has n customers to visit and wants to return home having driven as little as possible. This is the *travelling salesman problem* (TSP), one of the most studied problems of combinatorial optimisation and one of the first shown to be NP-hard [1]: no algorithm is known that solves every instance in time polynomial in n, and none is expected to exist. In practice one therefore settles for *heuristics*, which run fast and return good tours without any guarantee of optimality.

Origin of the project. The work began in 2023 as the term project of the course *Projet Informatique* [2]. Each pair of students chose one of three problems (DNA sequencing, delivery tours, scheduling with deadlines) and one of three generic heuristics (simulated annealing, a genetic algorithm, an ant colony), and was asked to:

(i) state the function to optimise;

(ii) choose a representation for the feasible solutions;

(iii) specify the problem-dependent parts of the method, here how an ant moves and how the probability of a move is computed;

(iv) implement the method in C and tune its parameters experimentally;

(v) evaluate and compare the results.

We chose delivery tours and the ant colony. The program we handed in worked, in the sense that it printed plausible tours, but it was a prototype: a single 300-line function, hard-coded parameters, and several bugs that happened not to show on the instances we tried.

This edition. Three years later the program has been rewritten from scratch as a reusable library with a test suite, and the evaluation, which in 2023 consisted of looking at pictures of tours, has been redone quantitatively. The report follows the five items above: Section 2 covers (i) and (ii), Section 3 covers (iii), Section 4 covers the implementation, and Section 5 covers the tuning and the evaluation. Section 6 concludes.

2 The problem

2.1 Statement

The customers and the depot are the vertices $1, \dots, n$ of a complete undirected graph. The edge between i and j carries a weight $d_{ij} \geq 0$, the distance (or the travel time) between the two places, with $d_{ij} = d_{ji}$ and $d_{ii} = 0$. A *tour* is a cycle that visits every vertex exactly once, and the problem is to find a tour of minimum total weight.

Nothing else is assumed about the distances. In particular they need not satisfy the triangle inequality: the instances supplied with the assignment are matrices of arbitrary integers. Instances built from points in the plane, where d_{ij} is the Euclidean distance, are an important special case and the only one that can be drawn.

2.2 Objective function

A tour is described by the order $\pi = (\pi_1, \ldots, \pi_n)$ in which it visits the cities, a permutation of $\{1, \ldots, n\}$. Its length, the function to minimise, is

$$L(\pi) \;=\; \sum_{i=1}^{n-1} d_{\pi_i \pi_{i+1}} \;+\; d_{\pi_n \pi_1}, \tag{1}$$

the last term being the trip back to the starting point. Rotating π or reversing it describes the same cycle, so there are $(n-1)!/2$ distinct tours: 181 440 for 10 cities, about 6×10^{16} for 20, and more than 10^{155} for 100. Enumeration is hopeless beyond a dozen cities.

2.3 Representation of a solution

A tour is stored exactly as π: an array of n city indices. This representation makes every operation the algorithm needs cheap. Its length is computed in $O(n)$ with Equation (1); an ant extends a partial tour by appending one index; and the 2-opt move of Section 3.3 is the reversal of a contiguous slice of the array. *Feasibility* is the condition that the array is a permutation, which the construction procedure guarantees by never proposing a city twice. When a tour is printed it is rotated to start at city 1, so that equal tours look equal.

The instance is stored as the full $n \times n$ matrix (d_{ij}) in one contiguous block, row after row. Symmetry would allow half of it to be saved, but the matrix is read far more often than anything else in the program, and d_{ij} at offset $(i-1)n + (j-1)$ is one multiplication away.

2.4 Input format

The assignment prescribes a text format: the number of cities, then the part of the matrix above the diagonal, row by row. For five cities:

```
5
61 789 204 349
362 135 376
260 45
880
```

so that $d_{12} = 61$, $d_{15} = 349$ and $d_{45} = 880$. The program also reads the common subset of TSPLIB [3], the format of the standard benchmark library, which can describe an instance by the coordinates of its cities.

3 Ant colony optimisation

3.1 Principle

Ants looking for food leave a chemical trail, a *pheromone*, and tend to follow the trails they come across. An ant that happens upon a short path to the food returns sooner, so its trail is reinforced sooner and attracts more ants, who reinforce it further; trails that are not maintained evaporate. No ant knows the terrain, yet the colony ends up using a short path: the trails are a shared memory that the ants both read and write.

Ant colony optimisation [4] turns this into an algorithm. Artificial ants repeatedly build solutions step by step, at random, with a bias towards the choices that earlier good solutions were made of. For the TSP a *step* is a move from the current city to one not visited yet, and

the pheromone is a number τ_{ij} attached to every edge. The first such algorithm, and the one implemented here, is *Ant System*, due to Dorigo, Maniezzo and Colorni [5].

3.2 Ant System

Moving an ant. The colony has m ants. At each iteration every ant builds a complete tour. Suppose ant k stands in city i and let N_i^k be the set of cities it has not visited. It moves to $j \in N_i^k$ with probability

$$p_{ij}^k \;=\; \frac{\tau_{ij}^{\alpha}\,\eta_{ij}^{\beta}}{\displaystyle\sum_{l \in N_i^k} \tau_{il}^{\alpha}\,\eta_{il}^{\beta}}, \qquad \text{where} \qquad \eta_{ij} = \frac{1}{d_{ij}}. \tag{2}$$

Two quantities make an edge attractive. Its *visibility* η_{ij} is fixed by the instance and says that near cities are good candidates: it is the greedy part of the decision. Its pheromone τ_{ij} changes over time and says that the edge belonged to good tours: it is the experience of the colony. The exponents α and β weigh one against the other. With $\alpha = 0$ the ants ignore each other and the algorithm is a randomised greedy search; with $\beta = 0$ they ignore the distances and can only amplify whatever the first random tours contained.

In practice the choice is made by a *roulette wheel*: the weights $\tau_{il}^{\alpha}\eta_{il}^{\beta}$ of the candidates are laid end to end on a segment, a point of the segment is drawn uniformly, and the candidate that owns it is selected.

Updating the pheromone. When all ants have completed their tours $T_1, \dots, T_m$, of lengths $L_1, \dots, L_m$, every trail first loses a fraction ρ of its pheromone and is then reinforced by the ants that used it:

$$\tau_{ij} \;\leftarrow\; (1-\rho)\,\tau_{ij} \;+\; \sum_{k=1}^{m} \Delta\tau_{ij}^k, \qquad \Delta\tau_{ij}^k = \begin{cases} Q/L_k & \text{if the edge } \{i,j\} \text{ belongs to } T_k, \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

An ant has a fixed budget Q of pheromone and spreads it over its tour, so the edges of a short tour receive more. Evaporation, governed by $\rho \in [0,1]$, is what lets the colony forget: without it the first tours, which are essentially random, would weigh on the search for ever.

Initialisation. The trails start from a common value τ_0. If it is much smaller than the first deposits, the first iteration decides everything; if it is much larger, many iterations pass before the deposits matter. We follow the recommendation of Dorigo and Stützle [4] and set

$$\tau_0 \;=\; \frac{m\,Q}{L_{\mathrm{nn}}}, \tag{4}$$

where L_{nn} is the length of a nearest-neighbour tour: this is the amount one iteration would deposit on an edge if all ants followed that tour, that is, the order of magnitude the trails will settle at. The 2023 program used the constant $\tau_0 = 0.2$, which is far too large for instances with long distances and too small for instances with short ones.

Algorithm 1 summarises the method. Each ant starts from its own city when $m = n$, as in the original paper, and from a random city otherwise.

Cost. An ant makes $n-1$ moves and examines the remaining cities at each of them, $n(n-1)/2$ candidates in total, so an iteration costs $O(m\,n^2)$ for the construction and $O(n^2 + m\,n)$ for the update. With one ant per city, the usual choice, this is $O(n^3)$ time per iteration and $O(n^2)$ memory.

Algorithm 1 Ant System for the travelling salesman problem

Require: distances (d_{ij}); parameters m, α, β, ρ, Q; number of iterations $t_{\max}$
Ensure: the best tour found, T^*

$\tau_{ij} \leftarrow \tau_0$ for every edge; $\quad L^* \leftarrow \infty$
for $t = 1, \dots, t_{\max}$ **do**
  for each ant $k = 1, \dots, m$ **do**
    place the ant in its starting city; mark every other city unvisited
    while some city is unvisited **do**
      choose the next city j with the probabilities p_{ij}^k of Equation (2)
      move to j and mark it visited
    end while
    $L_k \leftarrow$ length of the tour T_k just built, back to the start included
    if $L_k < L^*$ **then** $T^* \leftarrow T_k$; $\quad L^* \leftarrow L_k$
  end for
  $\tau_{ij} \leftarrow (1 - \rho)\,\tau_{ij}$ for every edge ▷ evaporation
  for each ant k and each edge $\{i, j\}$ of T_k **do**
    $\tau_{ij} \leftarrow \tau_{ij} + Q/L_k$ ▷ reinforcement
  end for
end for
return T^*

3.3 Three refinements

Ant System was soon improved upon. The program implements three ideas from that later work; each is off by default and can be enabled on its own.

Elitism. The best tour found so far, T^*, is reinforced at every iteration as if e additional ants had followed it [5]: the update (3) gains the term $e\,Q/L^*$ on the edges of T^*. This focuses the search around the incumbent and speeds up convergence, at the risk of converging too early.

Greedy exploitation. Ant Colony System [6] replaces the roulette wheel by the *pseudo-random proportional rule*: with probability q_0 the ant goes straight to the most attractive city, $j = \arg\max_{l \in N_i^k} \tau_{il}^{\alpha} \eta_{il}^{\beta}$, and with probability $1 - q_0$ it draws from Equation (2) as before. Setting $q_0 = 1$ reproduces the first variant we tried in 2023, which always took the most probable city; without randomness all ants soon follow the same tour, which is why we abandoned it.

Local search. A tour built by an ant usually contains obvious defects, such as two edges that cross. The 2-opt procedure [7] removes them: it looks for two edges $\{a, b\}$ and $\{c, d\}$ of the tour such that

$$d_{ac} + d_{bd} \;<\; d_{ab} + d_{cd},$$

replaces them by $\{a, c\}$ and $\{b, d\}$, which amounts to reversing the part of the tour between b and c, and repeats until no such pair exists. Applying it to every tour before the pheromone update turns the colony into a guided multi-start local search: the ants supply diverse, promising starting points and the trails remember which edges survive the improvement. This combination is the main ingredient of the best-performing ant algorithms for the TSP [4, 8].

3.4 Parameters

Table 1 lists the parameters, the options that set them and the defaults retained after the study of Section 5.3.

Table 1: Parameters of the algorithm.

Symbol	Option	Default	Role
m	`--ants`	n	number of ants
$t_{\max}$	`--iterations`	200	number of iterations
α	`--alpha`	1	weight of the pheromone
β	`--beta`	3	weight of the visibility
ρ	`--rho`	0.5	evaporation rate
Q	`--deposit`	1	pheromone budget of an ant
τ_0	`--tau0`	mQ/L_{nn}	initial pheromone
e	`--elitist`	0	weight of the best-so-far tour
q_0	`--q0`	0	probability of a greedy move
	`--local-search`	none	`none` or `2opt`
	`--patience`	off	iterations without progress before stopping

4 Implementation

4.1 Architecture

The code is a static library, `libaco`, and a command-line tool, `aco-tsp`, that is a thin layer on top of it. Figure 1 shows the modules and their dependencies, which form a hierarchy without cycles. The library is written in standard C11 and depends on nothing but the C library. It is developed with GCC and Clang, and avoids every construct that would prevent it from building with MSVC.

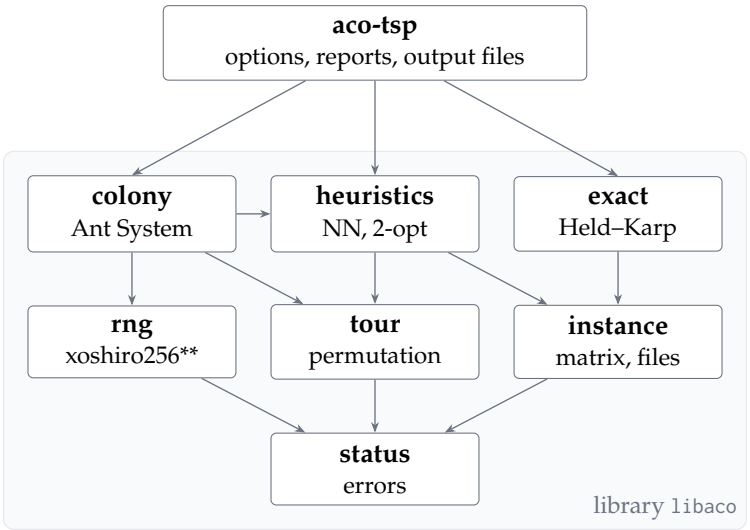


Figure 1: Modules and their main dependencies. An arrow from A to B means that A uses B.

Two kinds of types appear in the interface. An instance and a colony are *opaque handles*: they can only be created by functions that validate their input, so the rest of the code can rely

on a square, symmetric, finite, non-negative matrix with a zero diagonal without checking it again. Tours, parameters and statistics are plain structures. Every function that can fail returns a status code and, where an explanation is useful, fills in a message; the library never prints and never terminates the program. A complete use of the library is short:

```
aco_instance *instance = NULL;
aco_tour tour = ACO_TOUR_INIT;
aco_error error = ACO_ERROR_INIT;
aco_params params = aco_params_default();

params.local_search = ACO_LOCAL_SEARCH_TWO_OPT;
params.seed = 42;

if (aco_instance_load("cities.tsp", &instance, &error) == ACO_OK &&
    aco_solve(instance, &params, &tour, &error) == ACO_OK) {
    printf("best tour: %g\n", tour.length);
} else {
    fprintf(stderr, "error: %s\n", error.message);
}
aco_tour_free(&tour);
aco_instance_destroy(instance);
```

4.2 The inner loop

Almost all the running time is spent choosing the next city of an ant, so three decisions there matter.

Attractiveness is tabulated. The product $\tau_{ij}^{\alpha}\eta_{ij}^{\beta}$ of Equation (2) does not change while the ants of one iteration are moving. It is computed once per iteration for all edges, and η_{ij}^{β} once per run. A decision then costs additions and comparisons only; the 2023 code called `pow` twice for every candidate of every decision.

Unvisited cities are kept in a compact array. Visiting a city removes it by moving the last element of the array into its place, in constant time, so a decision only looks at cities that are still candidates. The 2023 code searched the partial route to find out whether a city had been visited, which made the construction of a single tour cubic in n.

Nothing is allocated. The colony allocates its buffers when it is created and reuses them at every iteration.

4.3 Numerical robustness

The formulas hide a few traps, which the implementation handles explicitly.

- The visibility is measured in units of the shortest positive distance of the instance, $\eta_{ij} = d_{\min}/d_{ij} \in (0, 1]$. Only ratios matter in Equation (2), and bounded values cannot overflow when raised to a large power β.
- Two cities at the same place have $d_{ij} = 0$. They receive the largest visibility instead of a division by zero.
- With extreme parameters every weight of a decision can underflow to zero. The ant then chooses uniformly among the candidates rather than dividing by a zero sum.

- In floating-point arithmetic two tours can each appear shorter than the other by a rounding error, and 2-opt could alternate between them for ever. A move is accepted only if it shortens the tour by more than a tiny fraction of the edges it removes.

4.4 Randomness and reproducibility

The roulette wheel of the 2023 program drew its point with `rand() % 100`, so the wheel had only a hundred positions. A candidate whose slice contained none of them could never be selected, and on an instance of 100 cities, where the average probability of a candidate is 1 %, that is the fate of many. The generator was also never seeded, and its sequence differs from one C library to another.

The library carries its own generator, xoshiro256** [9], seeded through SplitMix64. It is fast, passes the standard statistical tests, has an explicit state, so two colonies never disturb each other, and produces the same sequence on every platform. A run is therefore a deterministic function of the instance, the parameters and the seed, which makes results reproducible and failures debuggable.

4.5 Reference solvers

To judge the colony one needs something to compare it with. The library provides three solvers for that purpose.

Nearest neighbour. From a starting city, go to the closest unvisited city until none is left [10]. It is the simplest reasonable heuristic, runs in $O(n^2)$, and also provides the length L_{nn} of Equation (4).

Nearest neighbour followed by 2-opt. The local search of Section 3.3 applied to a greedy tour: a much stronger baseline.

Exact dynamic programming. The algorithm of Held and Karp [11] computes, for every set S of cities and every $j \in S$, the length $C(S, j)$ of the shortest path that leaves city 1, visits exactly S and ends in j:

$$C(\{j\}, j) = d_{1j}, \qquad C(S, j) = \min_{k \in S \setminus \{j\}} \bigl(C(S \setminus \{j\}, k) + d_{kj} \bigr).$$

The optimum is $\min_j C(\{2, \ldots, n\}, j) + d_{j1}$. With $O(n^2 2^n)$ time and $O(n\, 2^n)$ memory it is limited to 20 cities, where it takes a quarter of a second, but within that limit it provides the ground truth against which the heuristics are tested.

When used as baselines in Section 5, the two heuristics are run from each of the n possible starting cities and the best tour is kept, which gives them a budget comparable to one iteration of a colony with n ants.

4.6 From the 2023 prototype to version 2

Table 2 lists the defects of the original program and what replaced them. The first is the most serious: every array of N elements was indexed from 1 to N, so the program wrote past the end of each of them. It only appeared to work because the overwritten memory usually belonged to a neighbouring array.

Table 2: Defects of the 2023 program and their remedy in version 2.

2023 prototype	Version 2
Arrays of N elements indexed from 1 to N: out-of-bounds reads and writes	Zero-based indexing; whole test suite run under AddressSanitizer and Valgrind
Matrices in variable-length arrays on the stack: the instance size is limited by the stack	One heap block per matrix, allocated once and checked
Roulette wheel drawn with `rand() % 100`; generator not seeded	53-bit uniform draws from a seeded, portable generator
Membership test by scanning the route: $O(n^3)$ per ant	Compact array of unvisited cities: $O(n^2)$ per ant
Instance, parameters and output files fixed at compile time	Command-line options; nothing to recompile
Input files neither checked for existence nor validated	Parser with line-numbered diagnostics, fuzzed
Four copies of the same `main` function (1600 lines in total)	A library of independent modules, each with its tests
Plot through a pipe to a hard-coded Windows path	Portable data files and gnuplot scripts

4.7 Verification

A program that returns *a* tour is easy to write and hard to trust, since a wrong answer looks just like a mediocre one. The project therefore invests in several independent layers of verification.

- **Unit tests** (73, in five programs) exercise each module through its public interface, including the error paths. The exact solver is checked against brute-force enumeration; 2-opt is checked on points placed on a circle, where the optimum is the perimeter of a regular polygon; one iteration of the pheromone update is checked against the value of Equation (3) computed by hand.
- **Properties of the search** are tested rather than particular outputs: the same seed gives the same run, the best tour never gets worse, the trails stay symmetric and positive, and on small instances the result stays within a stated margin of the proven optimum.
- **End-to-end tests** (44 checks) run the command-line tool and verify its exit status, its messages and the files it writes.
- **Fuzzing.** The parser is the only code that reads untrusted data. A libFuzzer harness feeds it mutated files under AddressSanitizer and UndefinedBehaviorSanitizer; several million inputs are tried on every push.
- **Analysis.** The code compiles without warnings under a strict warning set with GCC and Clang, and passes clang-tidy with every finding treated as an error.

The tests cover 93.6 % of the lines and every function of the library and of the tool; what remains is almost entirely the handling of failed memory allocations. A continuous-integration pipeline repeats all of this for every change on Linux, macOS and Windows, and adds a CodeQL analysis.

5 Experiments

5.1 Protocol

Unless stated otherwise the colony has one ant per city and runs for 200 iterations with the default parameters of Table 1. Because the algorithm is randomised, every configuration is run with 20 different seeds and we report the mean length, the standard deviation and the best run. Three variants are compared: plain Ant System (AS), elitist Ant System with $e = n$, and Ant System with 2-opt.

Two families of instances are used: the seven matrices supplied with the assignment, of 5 to 100 cities, whose distances are arbitrary integers; and random Euclidean instances, generated by the program itself, with integer coordinates drawn uniformly from $[0, 1000]^2$ and distances rounded to the nearest integer. One of the latter, with 100 cities, serves for the tuning and the illustrations.

Times were measured on a laptop with an Apple M1 Pro processor, the program being compiled with Apple Clang 21 at `-O3` and running on a single core. All the data of this section can be regenerated with `scripts/run-experiments.sh`; the tour lengths are identical from one machine to another, only the timings change.

5.2 Validation against proven optima

The first question is whether the algorithm finds optimal tours when they are known. For each size from 8 to 16 cities we generated 100 random Euclidean instances, computed their optimum with the exact solver, and ran each variant once. Table 3 gives the proportion of instances solved to optimality and the relative excess over the optimum.

Table 3: Small random instances: frequency of reaching the proven optimum, and excess length over it.

		Ant System			Ant System + 2-opt	
Cities	Instances	Optimal	Mean excess	Worst excess	Optimal	Mean excess
8	100	100 %	0.00 %	0.0 %	100 %	0.00 %
10	100	94 %	0.13 %	5.7 %	100 %	0.00 %
12	100	95 %	0.07 %	3.2 %	100 %	0.00 %
14	100	89 %	0.09 %	1.7 %	100 %	0.00 %
16	100	89 %	0.14 %	5.2 %	100 %	0.00 %

Plain Ant System reaches the optimum in about nine runs out of ten and is on average within 0.15 % of it, although an unlucky run can end 5 % to 6 % above. With 2-opt, all 500 runs returned an optimal tour. Beyond their intrinsic interest, these figures are strong evidence that the implementation is correct.

5.3 Tuning the parameters

Figure 2 shows the effect of α, β and ρ on plain Ant System, each varied alone around the default configuration, on the random instance of 100 cities.

Both sources of information are necessary. Without pheromone ($\alpha = 0$) the mean tour is 44 % longer than with $\alpha = 1$; without visibility ($\beta = 0$) it is five times longer, barely better than a random tour. The greedy information matters most, but it is the pheromone that turns a randomised greedy search into an optimiser.

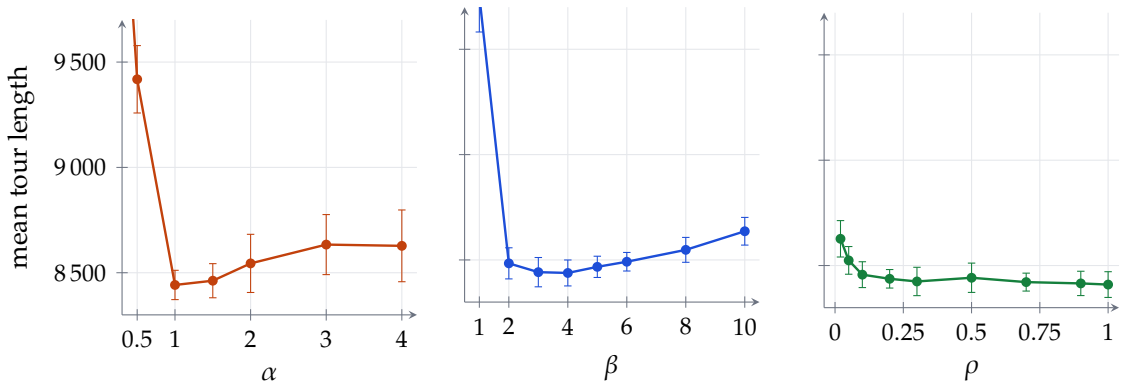


Figure 2: Sensitivity of Ant System to its three main parameters on 100 random cities. Each point is the mean of 20 runs and the bars show one standard deviation. The points $\alpha = 0$ (mean 12 123) and $\beta = 0$ (mean 41 867) lie far above the frame.

$\alpha = 1$ **is the best value.** Larger values make the ants follow the trails too faithfully: the mean degrades and, tellingly, the spread between runs doubles, the sign of a search that locks onto whichever tour came first.

β **has a broad optimum between 2 and 5.** The original program used $\beta = 1$ in one of its versions, which costs 16 % here.

ρ **hardly matters above 0.1.** Differences in that range are of the order of the noise. Very slow evaporation ($\rho \le 0.05$) costs one to two percent with a budget of 200 iterations: the early, poor tours are forgotten too slowly.

We retain $\alpha = 1$, $\beta = 3$ and $\rho = 0.5$, which agree with the values recommended in the literature [4].

5.4 Results on the instances of the assignment

Table 4: Tour lengths on the instances of the assignment. The baselines keep the best of n starting cities; the colony columns give the mean and the best of 20 runs of 200 iterations. The optimum is known for the first three instances only.

			Baselines		Ant System		Elitist AS		AS + 2-opt	
Instance	n	Optimum	NN	NN + 2-opt	mean	best	mean	best	mean	best
`tsp005`	5	64	64	64	64.0	64	64.0	64	64.0	64
`tsp010`	10	10	10	10	10.0	10	10.0	10	10.0	10
`tsp020`	20	201	274	210	213.2	201	205.6	201	201.0	201
`tsp030`	30	–	344	257	267.1	258	257.2	245	242.1	240
`tsp040`	40	–	374	272	296.1	288	284.2	271	264.1	263
`tsp050`	50	–	383	293	293.9	268	266.6	256	258.0	254
`tsp100`	100	–	3464	2801	3019.3	2892	2769.2	2573	2620.9	2552

Table 4 compares the three variants with the two baselines. Four observations can be made.

1. **The small instances are solved.** For 5, 10 and 20 cities the optimum is known; every variant finds it on the first two, and on `tsp020` the hybrid finds it in all 20 runs while plain Ant System does so in its best run.

2. **Ant System clearly beats the greedy heuristic**, by 13 % to 23 %, which is what the 2023 report observed qualitatively.
3. **Plain Ant System does not beat a good local search.** Nearest neighbour followed by 2-opt, restarted from every city, does as well or better on every instance, and does so in a few milliseconds. This sobering comparison is absent from the original report, which had no baseline at all. Elitism narrows the gap and overtakes the baseline on three of the five larger instances, including the two largest, where focusing the search pays off.
4. **The hybrid is the best method on every instance.** From 20 cities upwards its mean tour is 2.9 % to 11.9 % shorter than the stronger baseline, and it is also the most stable, its standard deviation being several times smaller than that of Ant System. The ants and the local search complement each other. The price is time: about half a second for 100 cities, where the baseline needs three milliseconds and plain Ant System 0.15 s.

5.5 Convergence

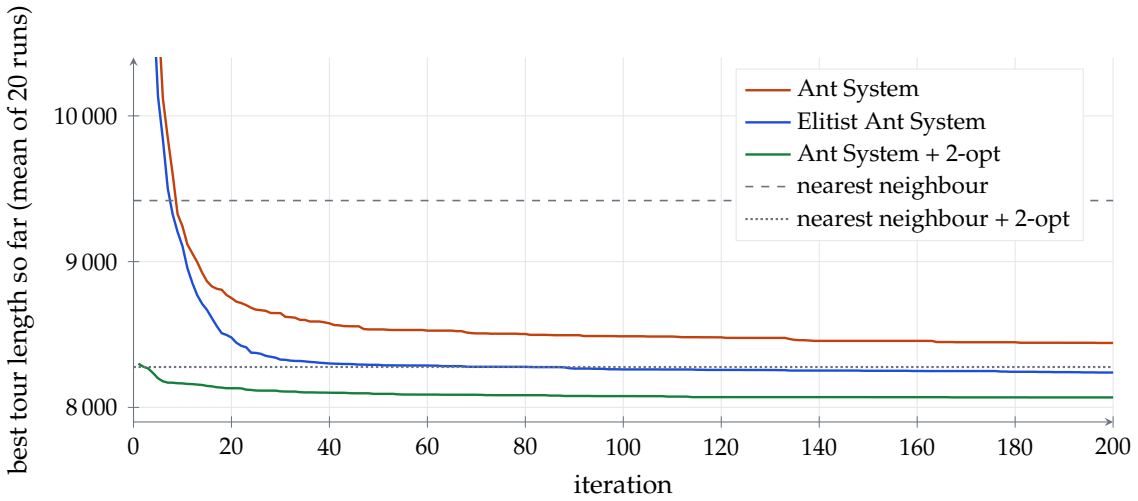
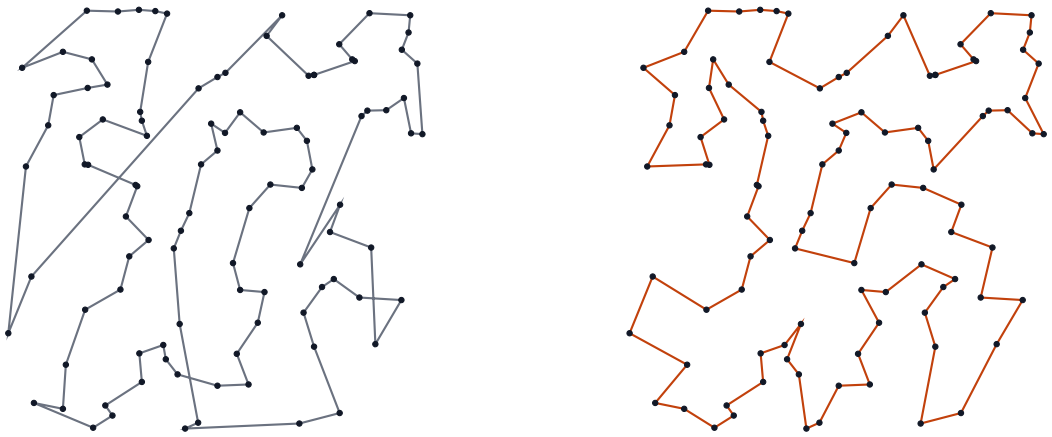


Figure 3: Convergence on 100 random cities. The first iterations of the two variants without local search, above 13 000, are cut off.

Figure 3 follows the best tour found so far on the random instance. Plain Ant System improves quickly during the first thirty iterations, when the trails take shape, and slowly afterwards; it ends at 8442.0 on average, well below the greedy tour (9419) but above the local-search baseline (8277). The elitist variant converges faster and lower, to 8239.2, just under that baseline. The hybrid is of another kind: its very first iteration, which is nothing but 100 randomised greedy tours improved by 2-opt, is already better than the final result of plain Ant System, and the trails then guide it down to 8068.4. All three curves flatten well before the end, which indicates that 200 iterations are sufficient at this size and that additional effort would be better spent on diversification than on more iterations.

Figure 4 shows what the numbers mean geometrically. The greedy tour is recognisable by the long edges it needs at the end to collect the cities it left behind; the tour of the hybrid has no crossing and no detour of that kind.



(a) Nearest neighbour: 9419 **(b)** Ant System + 2-opt: 8080

Figure 4: Two tours of the same 100 random cities.

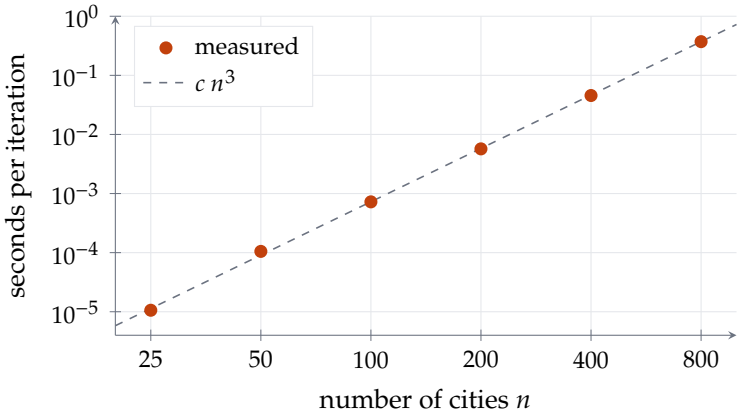


Figure 5: Time of one iteration of Ant System with one ant per city.

5.6 Running time

Figure 5 plots the time of one iteration against the size of the instance, with $m = n$. On logarithmic axes the measurements fall on a line of slope 3, as the analysis of Section 3.2 predicts: doubling the number of cities multiplies the time by eight. In absolute terms an iteration takes 0.72 ms for 100 cities and 0.37 s for 800, so the default 200 iterations take 0.15 s and about 75 seconds respectively. The local search multiplies these times by about four at 100 cities. The cubic growth is the practical limit of the method as implemented: instances of several thousand cities would require the candidate lists discussed below.

6 Conclusion

The five tasks of the assignment have been carried out. The objective is the length of a closed tour, a solution is a permutation, an ant moves according to the random proportional rule and the colony learns through evaporation and reinforcement. The algorithm is implemented as a tested C library, its parameters have been tuned by experiment, and its results have been compared with two heuristics and with an exact method.

The experiments support three conclusions. First, the implementation can be trusted: it finds proven optima on small instances. Second, the parameters behave as the literature describes, and the defaults $\alpha = 1$, $\beta = 3$, $\rho = 0.5$ are sound. Third, and less expected by the students we were in 2023, Ant System alone is not competitive with a simple local search; its value appears when the two are combined, the hybrid being the best method on every instance we tried.

The exercise of rewriting the program was instructive in its own right. Most of the defects of the first version were invisible in its output, and were found only because the new version was built to be checked: by tests, by sanitizers, by a fuzzer, and by baselines that could contradict us.

Further work. Several extensions follow directly from the limits observed. *Candidate lists*, which restrict each decision to the nearest few cities, would cut the cost of an iteration from $O(n^3)$ to $O(n^2)$ and open the way to much larger instances. *MAX–MIN Ant System* [8], which bounds the trails and lets only the best ant deposit, addresses the premature convergence visible in Figure 3. The ants of one iteration are independent and could build their tours *in parallel*. Finally, the delivery problem of the assignment mentions time windows for the customers, which would turn the construction step into a constrained one.

References

[1] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, New York, 1972. doi: 10.1007/978-1-4684-2001-2_9.

[2] Frédéric Roupin. Projet informatique. Course slides, CP2I2, Sup Galilée, Université Sorbonne Paris Nord, 2023. In French.

[3] Gerhard Reinelt. TSPLIB—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991. doi: 10.1287/ijoc.3.4.376.

[4] Marco Dorigo and Thomas Stützle. *Ant Colony Optimization*. MIT Press, Cambridge, MA, 2004. ISBN 978-0-262-04219-2.

[5] Marco Dorigo, Vittorio Maniezzo, and Alberto Colorni. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 26(1):29–41, 1996. doi: 10.1109/3477.484436.

[6] Marco Dorigo and Luca Maria Gambardella. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53–66, 1997. doi: 10.1109/4235.585892.

[7] G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6): 791–812, 1958. doi: 10.1287/opre.6.6.791.

[8] Thomas Stützle and Holger H. Hoos. MAX–MIN ant system. *Future Generation Computer Systems*, 16(8):889–914, 2000. doi: 10.1016/S0167-739X(00)00043-1.

[9] David Blackman and Sebastiano Vigna. Scrambled linear pseudorandom number generators. *ACM Transactions on Mathematical Software*, 47(4):36:1–36:32, 2021. doi: 10.1145/3460772.

[10] Daniel J. Rosenkrantz, Richard E. Stearns, and Philip M. Lewis, II. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, 1977. doi: 10.1137/0206041.

[11] Michael Held and Richard M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962. doi: 10.1137/0110015.

A Using the program

The program is built with CMake and has no dependency:

```
cmake --workflow --preset release
```

This configures the project, compiles it and runs the tests. A typical session then looks as follows.

```
$ build/release/aco-tsp solve data/random/random100.tsp --seed 1 --local-search 2opt
instance     random100 (100 cities)
algorithm    ant colony optimisation
parameters   ants=100 iterations=200 alpha=1 beta=3 rho=0.5 deposit=1
             tau0=0.009882 q0=0 elitist=0 local-search=2opt
seed         1
iterations   200 (best tour found at iteration 55)
time         0.582 s
length       8080
tour         1 2 58 54 12 4 38 13 18 78 80 26 39 77 69 81 65 53 64 84 15 74 9
             ...
```

Other useful invocations:

```
# Baselines and the exact solver
aco-tsp solve data/course/tsp020.txt --algorithm nearest-neighbour
aco-tsp solve data/course/tsp020.txt --algorithm two-opt
aco-tsp solve data/course/tsp020.txt --algorithm exact

# A new random instance, solved, with files for plotting
aco-tsp generate --cities 200 --seed 3 --output cities.tsp
aco-tsp solve cities.tsp --elitist 200 --plot tour.dat --trace trace.csv
gnuplot -e "input='tour.dat'; output='tour.svg'" scripts/plot-tour.gp

# Regenerate the data of every table and figure of this report
scripts/run-experiments.sh build/release/aco-tsp
```

The complete list of options is printed by `aco-tsp help solve`; the file formats are specified in `docs/file-formats.md` and the organisation of the code in `docs/architecture.md`.